

1. Concepto de programa informático

Definición y propósito · Tipos de programas · Programas y componentes del sistema

1.1 Definición y propósito

Un programa informático es un conjunto ORDENADO de instrucciones que, al ejecutarse en un ordenador, TRANSFORMA datos de entrada en resultados útiles para resolver un problema concreto.

Las tres palabras clave de la definición

Palabra clave	Qué significa
Conjunto de instrucciones	No es un único comando: es una secuencia de pasos, cada uno diciéndole al ordenador qué hacer. Puede haber decenas o millones.
Ordenado	El orden importa: si calculas la media antes de sumar las notas, el resultado es incorrecto. Gran parte de la dificultad de programar está en pensar la secuencia correcta.
Transforma entradas en salidas	Todo programa recibe algo, hace algo con ello y devuelve algo. Sin excepción.

Antes de escribir código, hazte siempre tres preguntas: ¿Qué datos recibo? ¿Qué tengo que hacer con ellos? ¿Qué resultado tengo que producir?

El modelo Entrada → Proceso → Salida



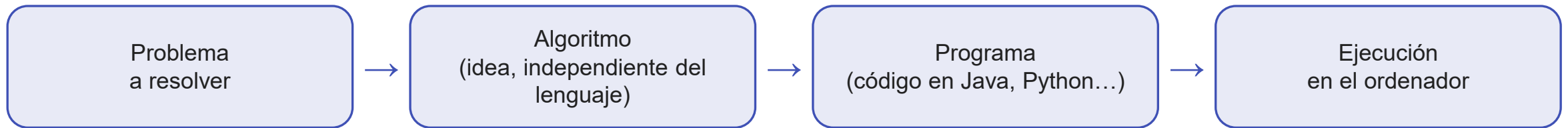
Entrada — lo que llega de fuera	Proceso — la lógica del programa	Salida — lo que produce
Lo que el usuario escribe en el teclado Un archivo leído del disco Un clic o un toque en pantalla Una señal de un sensor Una respuesta de una API externa	Cálculos matemáticos Comparaciones y decisiones (if, switch) Bucles y repeticiones Consultas a una base de datos Llamadas a otros programas o servicios	Un número o mensaje en pantalla Un archivo guardado en disco Una petición enviada a otro sistema Encender un LED o mover un motor Una página web devuelta al navegador

Ejemplo — calcular la nota media: Entrada → notas 6, 7, 8, 5, 9 · Proceso → sumar (35) y dividir entre 5 · Salida → "La nota media es 7,0". Si mañana las notas cambian, la entrada cambia pero el proceso (el código) es el mismo. El programa no inventa sus datos: los recibe.

La salida puede ser para el usuario, para otro programa o para un dispositivo físico.

Algoritmo vs. programa

Antes de escribir un programa hay que diseñar el algoritmo: el procedimiento lógico que resuelve el problema, sin preocuparse todavía del lenguaje de programación.



	Algoritmo	Programa
Qué es	El procedimiento lógico	La traducción a un lenguaje concreto
Depende del lenguaje	No	Sí
El ordenador lo ejecuta	No directamente	Sí
Ejemplo	"Recorre la lista y quédate con el mayor"	El código Java que lo implementa

Analogía: la receta de un pastel es el ALGORITMO (los pasos son los mismos en cualquier idioma u horno). El texto concreto escrito en español para un horno de gas sería el PROGRAMA.

Dos tipos de bug muy diferentes: el error puede estar en el ALGORITMO (la lógica es incorrecta) o en la IMPLEMENTACIÓN (la lógica es correcta pero está mal escrita). Se depuran de forma distinta — distinguirlos ahorra horas.

Ejemplo: encontrar el máximo de una lista

El algoritmo (independiente del lenguaje)

- Toma el primer elemento y guárdalo como "candidato a máximo".
- Recorre el resto de la lista de izquierda a derecha.
- Por cada elemento: ¿es mayor que el candidato actual? Sí → nuevo candidato. No → sigue.
- Al terminar, el candidato es el máximo. Funciona porque en todo momento el candidato es el mayor de los elementos vistos.

El programa en Java (cada paso del algoritmo = una línea)

```
public class MaximoLista {  
    public static int maximo(int[] a) {  
        int max = a[0];           // candidato inicial: el primero  
        for (int i = 1; i < a.length; i++) { // recorremos desde el segundo  
            if (a[i] > max) {         // ¿supera al candidato actual?  
                max = a[i];           // sí → nuevo candidato  
            }  
        }  
        return max;               // devolvemos el mayor  
    }  
}
```

Tres errores de concepto frecuentes

Error	Por qué es un error
Confundir datos de entrada con configuración	Los datos de entrada cambian en cada ejecución (las notas de cada alumno). La configuración son ajustes fijos del programa (el idioma, el número máximo de intentos). Mezclarlos hace el código confuso y difícil de mantener.
Creer que un programa necesita interfaz gráfica	Muchos programas funcionan solo en consola, como scripts o en segundo plano. Durante el módulo escribirás muchos sin ventanas — no son "menos programas".
Pensar que el programa controla el hardware directamente	Cuando tu código hace <code>System.out.println</code> no habla con el monitor: llama al sistema operativo, que gestiona los dispositivos. El programa trabaja sobre una capa de abstracción que ofrece el SO.

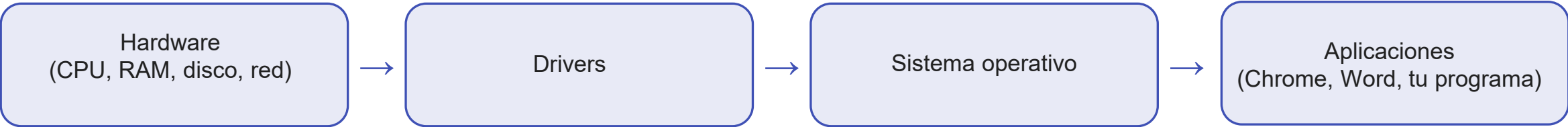
1.2 Tipos de programas — programas de sistema

Son el software más cercano al hardware. Gestionan los recursos físicos del ordenador y dan una plataforma estable para que se ejecute todo lo demás. El ejemplo más importante es el sistema operativo (Windows, Linux, macOS): es lo primero que se carga al encender.

Qué hace el sistema operativo

- Detecta el hardware disponible y lo carga en memoria.
- Decide qué proceso tiene acceso a la CPU en cada momento.
- Administra la RAM para que varios programas la usen sin pisarse.
- Controla el acceso al disco.
- Ofrece una API de llamadas al sistema que el resto del software usa.

Capa	Qué es
Drivers	Traducen instrucciones genéricas del SO a comandos del dispositivo concreto. Sin el driver, el SO no puede hablar con la impresora.
Firmware	Software grabado dentro del propio hardware (BIOS/UEFI, router, SSD). Viene de fábrica; actualizarlo es delicado.



El usuario final casi nunca interactúa directamente con los programas de sistema: están ahí para que todo lo demás funcione.

1.2 Tipos de programas — aplicaciones

Las aplicaciones son los programas que el usuario usa directamente para hacer algo concreto: escribir, navegar, editar fotos, jugar, gestionar una empresa. Hay tres subtipos con implicaciones técnicas distintas:

Subtipo	Dónde se ejecuta	Ventajas	Limitaciones	Ejemplos
Escritorio	Instalada en el ordenador, ejecución local	Acceso al sistema de archivos · usa todo el hardware · funciona sin internet	Hay que instalarla y actualizarla en cada equipo	Word, IntelliJ, Photoshop, VLC, Minecraft
Web	En un servidor; el usuario accede con el navegador	Accesible desde cualquier dispositivo · sin instalación	Requiere internet · sin acceso al disco local del usuario	Gmail, GitHub, campus virtual, tiendas online
Móvil	En smartphones o tablets	Acceso a sensores (GPS, cámara) · notificaciones push	Pantalla pequeña · recursos limitados · publicación en tienda (Play Store / App Store)	Instagram, Google Maps, la app del banco

DAW → desarrollarás principalmente aplicaciones WEB.

DAM → tu foco serán las aplicaciones MÓVILES.

Las diferencias entre tipos tienen consecuencias técnicas reales en cómo las diseñas.

1.2 Tipos de programas — utilidades y servicios

Utilidades: cuidar o configurar el sistema, no hacer el trabajo del día a día

Utilidad	Para qué sirve
Antivirus	Ver el estado del sistema, configurar análisis, eliminar amenazas
WinRAR / 7-Zip	Comprimir y descomprimir archivos
Administrador de Tareas	Ver procesos activos y consumo de recursos
GParted / CCleaner	Reorganizar particiones · limpiar archivos temporales

La línea aplicación/utilidad no siempre es nítida. La diferencia está en el propósito: ¿hacer el trabajo, o cuidar el sistema?

Servicios: programas en segundo plano, sin interfaz, esperando peticiones

No los abres tú: los inicia el SO al arrancar. Ejemplo — XAMPP instala Apache (el servidor web más usado del mundo), MySQL y PHP. Apache escucha en el puerto 80: el navegador pide GET http://localhost → Apache lee el archivo del disco → devuelve HTTP 200 + HTML. MySQL responde consultas en el puerto 3306. Un puerto es como una puerta numerada: cada servicio escucha en un número concreto para que el SO sepa a quién entregar cada petición.

Relevancia DAW/DAM: tus aplicaciones web se desplegarán como servicios. Tu código lo ejecuta un servidor que recibe peticiones HTTP las 24 horas, no un usuario haciendo doble clic. Compruébalo: Administrador de Tareas → pestaña Servicios → decenas de procesos activos sin que abras nada.

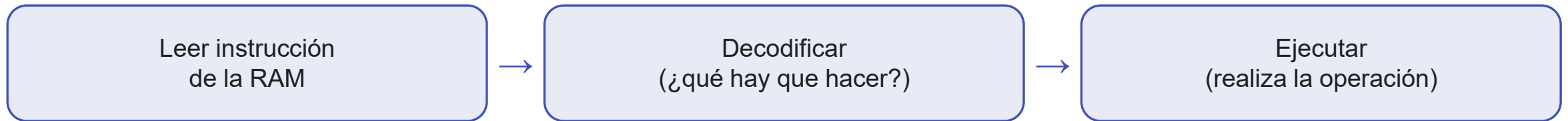
Resumen de los cuatro tipos

Tipo	Para qué sirve	¿Quién lo usa?	Ejemplo
Sistema	Gestiona el hardware y los recursos	El propio SO y otros programas	Windows, driver de la gráfica, BIOS
Aplicación	Permite al usuario realizar tareas	El usuario final	Chrome, Word, IntelliJ, Gmail
Utilidad	Mantenimiento y configuración del sistema	El técnico o usuario avanzado	Antivirus, WinRAR, Administrador de Tareas
Servicio	Atiende peticiones en segundo plano	Otros programas o equipos remotos	Apache, MySQL, Windows Update

1.3 Componentes del sistema — CPU (el procesador)

Principio clave: un programa NO accede directamente al hardware. Si tu código quiere leer un archivo, hace una petición al SO, que es quien tiene el control real y actúa de árbitro entre todos los programas.

La CPU ejecuta físicamente las instrucciones: cuando tu código recorre un bucle o evalúa un if, es la CPU quien lo hace a nivel de circuitos. Trabaja con un ciclo que se repite sin parar:



- Un procesador a 3 GHz realiza 3.000 millones de ciclos por segundo.
- Núcleos: varias CPUs independientes en el mismo chip. 8 núcleos = 8 tareas a la vez de verdad.
- Hilo: una "línea de montaje" de instrucciones que avanza de forma independiente. Si el programa solo tiene un hilo, usa un único núcleo aunque haya 8.

Consecuencia práctica: cada programa arranca con un hilo principal que gestiona la interfaz. Si pones ahí una operación pesada (recorrer un millón de registros, descargar un archivo grande), la app se CONGELA. La solución es moverla a un hilo secundario (programación concurrente, en módulos posteriores).

1.3 Componentes del sistema — memoria RAM

La RAM almacena el programa y sus datos MIENTRAS se ejecuta. El SO copia el programa del disco a la RAM, y la CPU trabaja con la RAM porque acceder a ella es mucho más rápido que al disco. Es VOLÁTIL: al apagar el ordenador, todo desaparece — guardar algo permanente exige escribirlo en disco explícitamente.

Analogía: la mesa de trabajo

Analogía	Informática
Cajón — donde guardas todo al terminar	Disco duro
Mesa — donde tienes lo que usas ahora	RAM
Documentos sobre la mesa — lo que procesas	Datos en memoria

Cuanto más grande la mesa (más RAM), más cosas abiertas a la vez. Cuando la mesa se llena, el SO usa el disco como RAM extra (memoria virtual): posible, pero muy lento — es cuando el ordenador va "a tirones".

Consecuencia práctica: cada objeto creado con `new` en Java vive en el heap, una zona de memoria que reserva la JVM (Java Virtual Machine, el programa que ejecuta tu código Java). Si cargas un millón de registros sin pensar, puedes consumir gigabytes; si superas la memoria disponible → `OutOfMemoryError` y el programa termina.

1.3 Componentes del sistema — almacenamiento y E/S

El disco: información permanente (código, .class, BD, configuración, logs)

	HDD (mecánico)	SSD SATA	SSD NVMe
Tecnología	Platos giratorios + cabezal	Chips flash (SATA)	Chips flash (PCIe, más rápido)
Velocidad lectura	~100–150 MB/s	~500 MB/s	~3.000–7.000 MB/s
Arranque del SO	30–60 s	10–15 s	5–10 s
Precio	Barato	Medio	Más caro

E/S: comunicación con el mundo exterior — y es LENTA (E/S bloqueante)

Operación	Tiempo aproximado
Acceder a la RAM	~100 nanosegundos
Leer del SSD / del HDD	~100 microsegundos (1.000× más lento) / ~10 ms (100.000×)
Respuesta de red local / servidor remoto	~1 ms / 50–200 ms
Esperar a que el usuario escriba	Segundos o más

1.3 Componentes del sistema — red

La red conecta el programa con otros equipos. Para un desarrollador web es el componente más relevante después de CPU y RAM: el navegador envía una petición HTTP al servidor, este la procesa (quizá consultando la base de datos) y devuelve la respuesta — en menos de 100 ms si todo va bien.

Latencia: el tiempo que tarda un dato en ir de un punto a otro

Trayecto	Latencia típica
Dentro de la misma máquina (localhost, el nombre con el que el SO se refiere a sí mismo)	< 1 ms
Servidor en el mismo datacenter (edificio con cientos de servidores)	1–5 ms
Servidor en otro país (Europa–USA)	80–150 ms
Servidor en otro continente	200–400 ms

50 llamadas de red en serie × 100 ms = 5 segundos para construir una página. Inaceptable en una aplicación web.

Fiabilidad: a diferencia de la CPU o la RAM, la red falla de forma intermitente e impredecible (servidor caído, conexión interrumpida, petición que no llega). Todo el código que usa la red debe gestionar errores: ¿qué hago si no hay respuesta en 5 segundos? ¿Qué muestro si el servidor está caído?

¿Qué ocurre cuando ejecutas un programa?

Al escribir `java MediaNotas` en la terminal, todos los componentes entran en juego:

Paso	Componente	Qué ocurre
1	Disco	El SO localiza <code>MediaNotas.class</code> y lo copia a la RAM
2	RAM	El programa y sus datos quedan cargados en memoria
3	CPU	La JVM (el programa que interpreta el bytecode de Java) ejecuta instrucción a instrucción
4	E/S (entrada)	Scanner espera a que escribas en el teclado; la CPU queda bloqueada
5	CPU	Calcula la media con los datos recibidos
6	E/S (salida)	<code>System.out.println</code> envía el resultado al monitor

Compruébalo: abre el Administrador de Tareas con el navegador abierto. Verás cientos de MB de RAM (pestañas en memoria), CPU intermitente (renderizado y JavaScript) y actividad de red en cada carga — todo gestionado por el SO en tiempo real.